

CITS5017 -- Rubric for Project 1 (2026)

Criteria	weight	Failed (0)	Novice (40)	Competent (70)	Proficient (100)
Data visualisation	12%	Not attempted or incomplete	Did not implement the function <code>display_images</code> correctly and required some modifications to run.	The function was mostly correct, taking in appropriate arguments and could run okay.	Outstanding code. Implementation clearly shows that some detailed thought had been put into the implementation. Function can be reused in later parts of the project. The code was neat, efficient, and explained well.
CNN (a) Network architecture	12%	Not attempted or incomplete	Implementation of the CNN architecture was not done correctly and had some issues. The code required some modifications to run.	The implementation of the CNN architecture followed the project specification. The code was mostly correct and ran without problems.	The implementation of the CNN architecture followed the project specification, with all the required components completed to a high standard. Code ran without problems. Code was also modularized and efficient, with clear structure. Appropriate markdown cells were provided and explanation was clear.
CNN (b) Hyperparameter tuning (including training)	20%	Not attempted or incomplete	Implementation of the hyperparameter tuning and training was not done correctly and had some issues. The code required some modifications to run.	The implementation of the hyperparameter tuning and training followed the project specification. The code was mostly correct and ran without problems.	The implementation of the hyperparameter tuning and training followed the project specification, with all the required components completed to a high standard. Code ran without problems. Code was also modularized and efficient, with clear structure. Appropriate markdown cells were provided and explanation was clear.
NASNetMobile (a) Model loading and visualisation	10%	Not attempted or incomplete	Implementation was not done correctly and had some issues. The code required some modifications to run.	The model loading code was done correctly. The code was mostly correct and ran without problems.	The model loading code was done correctly, with all the required components completed to a high standard. Code ran without problems. The code was neat. Appropriate markdown cells were provided and explanation was clear.
NASNetMobile (b) Network training	22%	Not attempted or incomplete	Implementation was not done correctly and had some issues. The code required some modifications to run.	The implementation followed the specification stated in the project. The model was trained and the code ran without problems.	The implementation followed the specification stated in the project. The model was trained and the code ran without problems. The code was neat. Graphics illustrations were good and easy to follow. Appropriate markdown cells were provided and explanation was clear.
Comparison (a) Overall performance of the two models	12%	Not attempted or incomplete	Implementation was not done correctly and had some issues. The code required some modifications to run.	All the required evaluation measures for comparing the two models were included. The Python code ran without problems.	All the required evaluation measures for comparing the two models were included. Graphics illustrations were good and easy to follow. The Python code was neat and efficient. Markdown cells were included and explanation was clear.
Comparison (b) Specific cases	12%	Not attempted or incomplete	Implementation was not done correctly and had some issues. The code required some modifications to run.	Specific cases were illustrated. Graphics illustrations were good and easy to follow. The Python code ran without problems. Some explanation was given in markdown cells.	Specific cases were illustrated. Graphics illustrations were good and easy to follow. The Python code ran without problems. The code was also neat and efficient. Markdown cells were included and explanation was clear.

100.00%

Penalty: If the two model files and/or the Jupyter Notebook file were not correctly named as stated in the project specification, a penalty of 5-10% will be applied, as we will need to rename your files manually before marking. **Do not email your submission to the Unit Coordinator** as a 10% penalty would be applied.

Late penalty is 5% per day, calculated on the hourly basis. Submissions within 48 hours after the due date incur no late penalty; however, after that, late penalty will be calculated from day 3 onward. Submissions overdue by 7 days or more will not be marked.

Note 1: we will only tick the “Proficient” column if **all** the required components stated in the project specification are fully implemented with good coding practice (modularized, neat, and efficient). Please check the project specification to make sure that you did not accidentally miss a required component, e.g., a missing learning curve plot or a missing model’s summary would mean that you would not get full mark for that criterion.

Note 2: For your markdown cells, ensure that you don’t provide just a heading and nothing else. Think about your Jupyter Notebook file as a document with code.

Note 3: Your code will be tested during the marking process. So, **make sure that it can run from beginning to end (bypassing the hyperparameter tuning part for the CNN model)**. A Jupyter Notebook file having code cells that are numbered in random order is considered to be messy.